

VI BOB. MASSIVLARNI SARALASH VA QIDIRISH (SORTING AND SEARCHING IN ARRAY)

NumPy-da saralash (**Sorting**) — bu massiv elementlarini ma'lum bir tartibda, ya'ni o'sish yoki kamayish tartibida joylashtirishni anglatadi. Buning uchun `np.sort()` funksiyasidan foydalaniladi. Odatiy holatda u elementlarni o'sish tartibida saralaydi, kamayish tartibiga esa `[::-1]` kabi slicing (kesib olish) usullari yordamida erishish mumkin.

6.1. NumPy massivlarini saralash (Sorting NumPy Array)

Massivni saralash ma'lumotlar tahlilida juda muhim bosqich hisoblanadi, chunki u ma'lumotlarni tartiblashga yordam beradi, qidirish va tozalash jarayonlarini osonlashtiradi.

Ushbu qo'llanmada biz NumPy-da massivni qanday saralashni o'rganamiz. Siz NumPy-da massivni quyidagicha saralashingiz mumkin:

- `np.sort()` funksiyasidan foydalangan holda (in-line saralash)
- Turli o'qlar (axes) bo'ylab saralash
- `np.argsort()` funksiyasidan foydalanish
- `np.lexsort()` funksiyasidan foydalanish

`sort()` funksiyasidan foydalanish

`sort()` metodi berilgan ma'lumotlar tuzilmasi (bu yerda massiv) elementlarini saralaydi. Elementlarni saralash uchun `sort` funksiyasini massiv obyekti bilan birga chaqiring.

`sort()` metodi yordamida massivni saralashning ikkita holati mavjud:

1. NumPy massivini o'z o'rnida (**in-place**) saralash
2. NumPy massivini o'qlar (**axes**) bo'ylab saralash

Biz quyida ushbu usullarning har ikkalasini misol bilan ko'rib chiqamiz:

Massivni o'z o'rnida (In-Place) saralash

Massivni o'z o'rnida saralash — bu asl massiv elementlarini to'g'ridan-to'g'ri saralashni anglatadi. U massivning yangi nusxasini yaratmaydi va xotira jihatidan juda samarali hisoblanadi.

Misol: NumPy massividagi elementlarni o'z o'rnida saralash uchun `sort()` metodidan foydalanish.

```
# kutubxonalarni yuklash
import numpy as np

a = np.array([12, 15, 10, 1])
print("Saralashdan oldingi massiv:", a)

# o'z o'rnida saralash
a.sort()
print("Saralashdan keyingi massiv:", a)
```

Saralashdan oldingi massiv: [12 15 10 1]

Saralashdan keyingi massiv: [1 10 12 15]

Izoh: E'tibor bering, `a.sort()` metodi chaqirilgandan so'ng `a` o'zgaruvchisining asl qiymati o'zgardi. Bu xotirani tejash uchun foydalidir, chunki yangi massiv obyekti yaratilmaydi.

Massivni turli o'qlar (Axes) bo'ylab saralash

Ushbu usul berilgan NumPy massivining saralangan nusxasini yaratadi. Bu ko'pincha ko'p o'lchamli massivlarda ma'lum bir o'lcham bo'ylab saralash kerak bo'lganda qo'llaniladi.

Misol: NumPy massivi elementlarini o'qlar (axis) bo'ylab saralash uchun `sort()` funksiyasidan foydalanish.

```
# Birinchi o'q (axis = 0) bo'ylab saralash (ustunlar bo'yicha)
a = np.array([[12, 15], [10, 1]])
arr1 = np.sort(a, axis = 0)
print("Birinchi o'q bo'ylab:\n", arr1)

# Oxirgi o'q (axis = -1) bo'ylab saralash (qatorlar bo'yicha)
a = np.array([[10, 15], [12, 1]])
arr2 = np.sort(a, axis = -1)
print("\nOxirgi o'q bo'ylab:\n", arr2)

# Hech qanday o'qsiz (axis = None) saralash (yassilangan holda)
a = np.array([[12, 15], [10, 1]])
arr3 = np.sort(a, axis = None)
print("\no'qsiz saralash:\n", arr3)
```

Birinchi o'q bo'ylab:

```
[[10  1]  
 [12 15]]
```

Oxirgi o'q bo'ylab:

```
[[10 15]  
 [ 1 12]]
```

o'qsiz saralash:

```
[ 1 10 12 15]
```

Izoh:

- **axis = 0** : Elementlar ustunlar bo'yicha saralanadi. Ya'ni har bir ustunning ichidagi qiymatlar tepadan pastga qarab o'sib boradi.
- **axis = -1** (yoki **axis = 1** 2D massiv uchun): Elementlar qatorlar bo'yicha saralanadi. Har bir qator ichidagi qiymatlar chapdan o'ngga qarab tartiblanadi.
- **axis = None** : Massiv avval yassilanadi (1D holatiga keltiriladi) va barcha elementlar yaxlit holatda saralanadi.

argsort() funksiyasidan foydalanish

argsort() metodi NumPy massivini berilgan o'q bo'ylab saralashning bilvosita usulidir. U asl massivni o'sish tartibida saralash uchun kerak bo'ladigan **indekslar massivini** qaytaradi.

Misol: NumPy massivi elementlarini saralash uchun **argsort()** dan foydalanish.

```
# NumPy massivini yaratish  
a = np.array([9, 3, 1, 7, 4, 3, 6])  
  
# Saralanmagan massivni chop etish  
print('Asl massiv:\n', a)  
  
# Massiv indekslarini saralash  
b = np.argsort(a)  
print('Asl massivning saralangan indeksleri ->', b)  
  
# Saralangan indekslar yordamida saralangan massivni olish  
# c - b bilan bir xil uzunlikdagi vaqtinchalik massiv  
c = np.zeros(len(b), dtype = int)  
for i in range(0, len(b)):  
    c[i] = a[b[i]]
```

```
print('Saralangan massiv ->', c)
```

Asl massiv:

```
[9 3 1 7 4 3 6]
```

Asl massivning saralangan indeksleri -> [2 1 5 4 6 3 0]

Saralangan massiv -> [1 3 3 4 6 7 9]

Izoh:

- `np.argsort(a)` funksiyasi bizga elementlarning o'zini emas, balki ularning tartiblangan o'rnini (indeksini) beradi. Masalan, eng kichik element `1` bo'lib, u asl massivda `2` -indeksda turibdi, shuning uchun natijaviy indekslar massivining birinchi elementi `2` ga teng.
- Kodning oxirgi qismida saralangan indekslar yordamida qiymatlarni qayta terib chiqish orqali saralangan massiv hosil qilinadi.
- Aslida, NumPy-da saralangan massivni olish uchun sikl (loop) ishlatish shart emas, buni **fancy indexing** yordamida osonroq bajarish mumkin: `a[b]`.

Kalitlar ketma-ketligidan foydalanish

Kalitlar ketma-ketligi yordamida saralash bizga massivni bir nechta mezonlar asosida tartiblash imkonini beradi. Ushbu usulni `np.lexsort()` funksiyasi orqali amalga oshirish mumkin. `lexsort()` funksiyasi asl massivni saralash uchun kerak bo'ladigan indekslar massivini qaytaradi.

Misol: Kalitlar ketma-ketligi yordamida barqaror saralashni amalga oshirish.

```
# Birinchi ustun
a = np.array([9, 3, 1, 3, 4, 3, 6])
# Ikkinchi ustun
b = np.array([4, 6, 9, 2, 1, 8, 7])

print('a ustuni, b ustuni')
for (i, j) in zip(a, b):
    print(i, ' ', j)

# Avval "a" bo'yicha, keyin esa "b" bo'yicha saralash
# Eslatma: Lexsort oxirgi kalit bo'yicha asosiy saralashni bajaradi
ind = np.lexsort((b, a))
```

```
print('Saralangan indekslar ->', ind)
```

```
a ustuni, b ustuni
```

```
9      4
```

```
3      6
```

```
1      9
```

```
3      2
```

```
4      1
```

```
3      8
```

```
6      7
```

```
Saralangan indekslar -> [2 3 1 5 4 6 0]
```

Izoh: `np.lexsort((b, a))` funksiyasida eng oxirgi ko'rsatilgan massiv (`a`) asosiy saralash kaliti hisoblanadi. Agar `a` massivida bir xil qiymatlar bo'lsa (masalan, 3 soni bir necha bor kelgan), u holda tartibni aniqlash uchun `b` massividagi mos qiymatlarga qaraladi. Bu xuddi Excel-dagi ko'p darajali saralashga o'xshaydi.

Xulosa

NumPy massivini saralash takrorlanuvchi elementlarni, maksimal va minimal qiymatlarni topishni osonlashtiradi. Bu ma'lumotlar bilan ishlashda juda muhim operatsiya bo'lib, ma'lumotlar manipulyatsiyasini sezilarli darajada soddalashtiradi.

Ushbu qo'llanmada biz NumPy-da massivni saralashning uchta usulini ko'rib chiqdik: `sort()`, `argsort()` va `lexsort()`. Bu usullarning barchasi NumPy-dagi `ndarray` ob'ektlarini saralash uchun turlicha funktsionalliklarni taqdim etadi. Mavzuni to'liq tushunishingiz uchun biz ushbu usullarni oddiy so'zlar va misollar bilan tushuntirib berdik.

6.2. Saralash texnikalarining turlari (Different Types of Sorting Techniques)

Ushbu saralash usullari bir-biridan sezilarli darajada farq qiladi. Keling, har bir texnika qanday ishlashini va qaysi birini tanlash kerakligini o'rganamiz.

Faraz qilaylik, `a` — bu NumPy massivi bo'lsin:

`a.sort()` metodi

- o'z o'rnida saralaydi (In-place):** Massivni to'g'ridan-to'g'ri o'zgartiradi va `None` qiymatini qaytaradi.
- Qaytarish turi:** `None` (ya'ni hech qanday yangi obyekt qaytarmaydi).

3. **Xotira tejamkorligi:** Kam joy egallaydi. Asl massivning nusxasi yaratilmagani uchun xotira sarfi minimal bo‘ladi.
4. **Tezkorlik:** Python-ning standart `sorted(a)` funksiyasiga qaraganda ancha tezroq ishlaydi.

Misol:

`a.sort()` yordamida massivni o‘z o‘rnida saralash.

```
# NumPy massivini yaratish
a = np.array([9, 3, 1, 7, 4, 3, 6])

# Saralanmagan massivni chop etish
print('Asl massiv:\n', a)

# Qaytarish turini tekshirish (None bo‘lishi kerak)
print('Qaytarish turi:', a.sort())

# Saralangan asl massiv natijasi
print('Saralangan asl massiv ->', a)
```

Asl massiv:

[9 3 1 7 4 3 6]

Qaytarish turi: None

Saralangan asl massiv -> [1 3 3 4 6 7 9]

Izoh: `a.sort()` metodi ishlatilganda, siz yangi o‘zgaruvchiga ehtiyoj sezmaydiz. Bu usul ayniqsa juda katta hajmdagi ma’lumotlar (Big Data) bilan ishlaganda xotira (RAM) yetishmovchiligini oldini olish uchun eng ma’qul yo‘ldir. Biroq, ehtiyot bo‘ling: agar sizga asl (saralanmagan) ma’lumotlar keyinchalik kerak bo‘lsa, bu metod ularni butunlay o‘zgartirib yuboradi.

`sorted(a)` funksiyasi

`sorted()` — bu Python-ning o‘rnatilgan (built-in) funksiyasi bo‘lib, u NumPy massivlari bilan ham ishlaydi, biroq uning ishlash tamoyili `a.sort()` metodidan tubdan farq qiladi.

1. **Yangi nusxa yaratadi:** Eski massiv asosida yangi ro‘yxat yaratadi va uni saralangan holda qaytaradi.
2. **Qaytarish turi:** Har doim **list (ro‘yxat)** qaytaradi (hatto kiruvchi ma’lumot NumPy massivi bo‘lsa ham).
3. **Xotira sarfi:** Ko‘proq joy egallaydi, chunki asl massivning nusxasi yaratiladi va shundan so‘ng saralanadi.

4. **Tezkorlik:** NumPy-ning ichki `a.sort()` metodiga qaraganda sekinroq ishlaydi.

Misol: `sorted()` yordamida saralangan nusxa yaratish.

```
# NumPy massivini yaratish
a = np.array([9, 3, 1, 7, 4, 3, 6])

# Saralanmagan massivni chop etish
print('Asl massiv:\n', a)

# Saralangan nusxa b o'zgaruvchisiga saqlanadi
b = sorted(a)

# b - saralangan ro'yxat, turi <class 'list'>
print('Yangi saralangan ro'yxat (b) ->', b)

# Asl massiv o'zgarishsiz qoladi
print('Asl massiv (a) ->', a)
```

Asl massiv:

[9 3 1 7 4 3 6]

Yangi saralangan ro'yxat (b) -> [np.int64(1), np.int64(3), np.int64(3), np.int64(4), np.int64(6), np.int64(7), np.int64(9)]

Asl massiv (a) -> [9 3 1 7 4 3 6]

Izoh:

`sorted(a)` funksiyasining eng katta afzalligi shundaki, u **asl ma'lumotlarni saqlab qoladi**. Agar sizga ham saralangan, ham dastlabki tartibdagi ma'lumotlar bir vaqtda kerak bo'lsa, ushbu funksiyadan foydalanish maqsadga muvofiqdir. Biroq, natija NumPy massivi emas, balki standart Python ro'yxati (`list`) bo'lib qaytishini yodda tutishingiz kerak. Agar sizga yana massiv kerak bo'lsa, natijani `np.array(b)` orqali o'girib olishingizga to'g'ri keladi.

`np.argsort(a)` funksiyasi

`np.argsort()` — bu massivni bilvosita saralash usuli bo'lib, u qiymatlarning o'zini emas, balki ularni tartibga soluvchi indekslarni qaytaradi.

1. **Indeksdlarni qaytaradi:** Massivni o'sish tartibida saralash uchun kerak bo'ladigan indekslar ketma-ketligini aniqlaydi.
2. **Qaytarish turi:** NumPy massivi (`ndarray`) qaytaradi.

3. **Xotira sarfi:** Yangi indekslar massivi yaratilgani uchun qo‘shimcha xotira egallaydi.

Misol: `np.argsort()` yordamida indekslar orqali saralash.

```
# NumPy massivini yaratish
a = np.array([9, 3, 1, 7, 4, 3, 6])

# Saralanmagan massivni chop etish
print('Asl massiv:\n', a)

# Massiv indekslarini saralash
b = np.argsort(a)
print('Asl massivning saralangan indeksleri ->', b)

# Saralangan indekslar yordamida saralangan massivni olish
# c - b bilan bir xil uzunlikdagi vaqtinchalik massiv
c = np.zeros(len(b), dtype = int)
for i in range(0, len(b)):
    c[i] = a[b[i]]

print('Saralangan massiv ->', c)
```

Asl massiv:

[9 3 1 7 4 3 6]

Asl massivning saralangan indeksleri -> [2 1 5 4 6 3 0]

Saralangan massiv -> [1 3 3 4 6 7 9]

Izoh: `np.argsort()` metodining eng foydali tomoni shundaki, u bizga bir nechta bog‘liq massivlarni bitta kalit bo‘yicha saralash imkonini beradi. Masalan, sizda talabalar ismlari va ularning ballari alohida massivlarda bo‘lsa, ballar bo‘yicha `argsort()` qilib, olingan indekslar yordamida ismlar massivini ham ballarga mos ravishda tartiblashingiz mumkin.

Maslahat: Yuqoridagi misolda `for` sikli o‘rniga NumPy-ning "**fancy indexing**" imkoniyatidan foydalanish ancha samarali: `c = a[b]` — bu bitta qator kod sikl bilan bir xil natijani beradi va ancha tezroq ishlaydi.

`np.lexsort((b, a))` funksiyasi

`np.lexsort()` — bu bir nechta kalitlar (massivlar) ketma-ketligi yordamida bilvosita saralashni amalga oshiradi. Bu usul ma’lumotlar bazasidagi bir nechta ustun bo‘yicha tartiblashga (masalan, familiyasi bir xil bo‘lganlarni ismi bo‘yicha saralashga) o‘xshaydi.

1. **Kalitlar ketma-ketligi bo'yicha bilvosita saralash:** Bir nechta mezon asosida indekslarni aniqlaydi.
2. **Ketma-ketlik tartibi:** `np.lexsort((b, a))` funksiyasida **oxirgi** ko'rsatilgan massiv (`a`) asosiy saralash kaliti bo'ladi, undan oldingisi (`b`) esa yordamchi kalit hisoblanadi.
3. **Qaytarish turi:** Indekslerden iborat `ndarray` (butun sonli massiv) qaytaradi.
4. **Xotira sarfi:** Juftliklar bo'yicha hisoblangan yangi indekslar massivi yaratilgani uchun qo'shimcha xotira egallaydi.

Misol: `np.lexsort()` yordamida bir nechta massiv bo'yicha saralash.

```
# NumPy massivlarini yaratish
a = np.array([9, 3, 1, 3, 4, 3, 6]) # Birinchi ustun (Asosiy kalit)
b = np.array([4, 6, 9, 2, 1, 8, 7]) # Ikkinchi ustun (Yordamchi kalit)

print('a ustuni, b ustuni')
for (i, j) in zip(a, b):
    print(i, ' ', j)

# Avval a bo'yicha, a lar teng bo'lsa b bo'yicha saralash
ind = np.lexsort((b, a))

print('Saralangan indekslar ->', ind)
```

```
a ustuni, b ustuni
9      4
3      6
1      9
3      2
4      1
3      8
6      7
Saralangan indekslar -> [2 3 1 5 4 6 0]
```

Izoh:

Natijani tahlil qilsak, nima uchun bunday indekslar chiqqanini tushunish oson:

- `a` massivdagi eng kichik qiymat `1` (indeksi `2`). U birinchi bo'lib chiqadi.
- Keyingi kichik qiymat `3`. Lekin `3` soni `1`, `3` va `5`-indekslarda joylashgan.

- Endi NumPy `b` massiviga qaraydi: `b[1]=6` , `b[3]=2` , `b[5]=8` .
- Ularning ichida eng kichigi `2` , shuning uchun keyingi indeks `3` , keyin `1` (chunki `6 < 8`) va keyin `5` bo'ladi.

Bu usul murakkab jadvallarni tartibga solishda juda qo'l keladi.

NumPy-da qidirish (**Searching**) — bu massiv ichidan muayyan qiymatlarni yoki shartlarni topishni anglatadi. Ushbu bo'limda biz NumPy massivlarida qidirishning turli usullarini, jumladan, `np.where()` orqali aniq qiymatlarni topish, `np.searchsorted()` va barcha nolga teng bo'lmagan elementlar indekslarini qaytaruvchi `np.nonzero()` funksiyalarini ko'rib chiqamiz.

6.3. NumPy massivlarida qidiruv amallari (Searching in NumPy Array)

NumPy har xil turdagi sonli qiymatlarni qidirish uchun turli usullarni taqdim etadi, ushbu maqolada biz ulardan ikkita muhimini ko'rib chiqamiz: `numpy.where()` va `numpy.searchsorted()` .

1. `numpy.where()`

Ushbu funksiya berilgan shart bajarilgan kiruvchi massiv elementlarining indekslarini qaytaradi.

Sintaksis: `numpy.where(shart[, x, y])`

Parametrlar:

- **condition (shart):** Shart rost (`True`) bo'lganda `x` qiymatini, aks holda `y` qiymatini beradi.
- **x, y:** Tanlash uchun qiymatlar. `x` , `y` va `shart` ma'lum bir shaklga mos kelishi (broadcastable) kerak.

Qaytariladigan qiymat:

- **out:** [ndarray yoki ndarray'lar korteji] Agar `x` va `y` ko'rsatilgan bo'lsa, chiqish massivi shart rost bo'lgan joylarda `x` elementlarini, boshqa joylarda esa `y` elementlarini o'z ichiga oladi.
- Agar faqat shart berilgan bo'lsa, `condition.nonzero()` korteji, ya'ni shart rost bo'lgan indekslar qaytariladi.

Misol: Quyidagi misol `where()` yordamida qanday qidirishni namoyish etadi.

```
# massiv yaratish
arr = np.array([10, 32, 30, 50, 20, 82, 91, 45])

# arr ni chop etish
print("arr = {}".format(arr))

# arr ichidan 30 qiymatini qidirish va uning indeksini i da saqlash
i = np.where(arr == 30)

print("i = {}".format(i))

arr = [10 32 30 50 20 82 91 45]
i = (array([2]),)
```

Ko‘rib turganingizdek, `i` o‘zgaruvchisi qidirilgan qiymatimizning indeksini birinchi element sifatida saqlovchi takrorlanuvchi obyekt (iterable) hisoblanadi. Oxirgi chop etish (print) qismini quyidagiga almashtirish orqali uni chiroyliroq ko‘rinishga keltirishimiz mumkin:

```
print("i = {}".format(i[0]))
```

Bu yakuniy natijani quyidagicha o‘zgartiradi:

```
arr = [10 32 30 50 20 82 91 45]
i = [2]
```

Izoh: `np.where()` nafaqat bitta qiymatni, balki murakkab shartlarni ham qidira oladi. Masalan, `np.where(arr > 40)` barcha 40 dan katta elementlar indeksini qaytaradi. Bu funksiya ma’lumotlarni filtrlash va shartli o‘zgartirishlar uchun juda qulaydir.

2. `numpy.searchsorted()`

Ushbu funksiya saralangan `arr` massivida ko‘rsatilgan qiymatlar tartibni buzmagun holda qaysi indeksga joylashtirilishi kerakligini aniqlash uchun ishlatiladi. Kerakli indekslarni topishda **ikkilik qidiruv (binary search)** algoritmidan foydalaniladi.

Sintaksis:

```
numpy.searchsorted(arr, num, side='left', sorter=None)
```

Parametrlar:

- **arr:** [array_like] Kiruvchi massiv. Agar `sorter` berilmagan bo‘lsa, u o‘zish tartibida saralangan bo‘lishi kerak.

- **num:** [array_like] Massivga joylashtirmoqchi bo'lgan qiymat(lar).
- **side:** ['left', 'right'], ixtiyoriy. Agar 'left' bo'lsa, topilgan birinchi mos joyning indeksi beriladi. Agar 'right' bo'lsa, oxirgi shunday indeks qaytariladi.
- **sorter:** [array_like, ixtiyoriy] Massivni o'sish tartibida saralovchi butun sonli indekslar massivi (odatda `argsort` natijasi).

Qaytaradi: [indices] `num` bilan bir xil shakldagi joylashtirish nuqtalari (indekslar) massivi.

Misol: Quyidagi misol `searchsorted()` funksiyasidan foydalanishni tushuntiradi.

```
# massiv yaratish
arr = [1, 2, 2, 3, 3, 3, 4, 5, 6, 6]
print("arr = {}".format(arr))

# Eng chapdagi 3 (birinchi uchrashi mumkin bo'lgan joy)
print("Eng chapdagi indeks = {}".format(np.searchsorted(arr, 3, side="left")))

# Eng o'ngdagi 3 (oxirgi uchrashi mumkin bo'lgan joy)
print("Eng o'ngdagi indeks = {}".format(np.searchsorted(arr, 3, side="right")))

arr = [1, 2, 2, 3, 3, 3, 4, 5, 6, 6]
Eng chapdagi indeks = 3
Eng o'ngdagi indeks = 6
```

Izoh:

- `side="left"` bo'lganda, funksiya massivdagi birinchi uchragan 3 ning o'rnini (3-indeks) qaytaradi.
- `side="right"` bo'lganda esa, oxirgi 3 dan keyingi o'rinni (6-indeks) qaytaradi, chunki yangi 3 ni o'sha yerga qo'ysak ham tartib saqlanib qoladi.
- Bu funksiya juda katta hajmdagi saralangan ma'lumotlar bilan ishlashda oddiy qidiruvga qaraganda ancha samaralidir.

6.4. NumPy massivlarida qidirish, saralash va sanash (Searching, Sorting, and Counting)

NumPy — bu sonli hisoblashlar uchun ishlatiladigan Python kutubxonasi bo'lib, u massivlar bilan bog'liq amallarni tez va samarali bajarishga imkon beradi. U massivlarni saralash, muayyan elementlarni qidirish va qiymatlar

uchrash sonini hisoblash uchun o'rnatilgan funksiyalarni taqdim etadi. Ushbu funksiyalar ma'lumotlar manipulyatsiyasi va tahlilini soddalashtirishga yordam beradi, bu esa katta ma'lumotlar to'plamlarida murakkab amallarni osonlik bilan bajarish imkonini beradi.

Saralash (Sorting)

NumPy-da saralash massiv elementlarini ma'lum bir tartibda, masalan, sonlar o'sishi yoki alifbo tartibida joylashtirishni anglatadi. Bu ma'lumotlar tashkil etilishini yaxshilaydi hamda qidirish va tahlil qilish jarayonlarini samaraliroq qiladi, ayniqsa katta hajmdagi yoki ko'p o'lchamli massivlar bilan ishlashda juda qo'l keladi.

1. `numpy.sort()`

`numpy.sort()` metodi asl massivni o'zgartirmagan holda kiruvchi massivning saralangan nusxasini qaytaradi. Bu asl ma'lumotlarni o'zgarishsiz saqlab, saralangan ma'lumotlarga ega bo'lishni xohlaganingizda foydalidir.

Misol:

```
arr = np.array([5, 2, 9, 1, 7])
sorted_arr = np.sort(arr)

print(sorted_arr)
```

```
[1 2 5 7 9]
```

Izoh: `np.sort()` funksiyasi o'zgaruvchan bo'lmagan (immutable) yondashuvni afzal ko'radigan holatlar uchun mos keladi. Agar sizga asl massivning o'zini saralash kerak bo'lsa, yuqorida ko'rib chiqqanimizdek `arr.sort()` metodidan foydalanishingiz mumkin.

NumPy-ning qidiruv va hisoblash funksiyalari bo'yicha yana qanday ma'lumotlar kerak?

2. `numpy.argsort()`

`numpy.argsort()` funksiyasi saralangan qiymatlarning o'zini emas, balki massivni saralash uchun kerak bo'ladigan **indekslarni** qaytaradi. Bu usul ko'pincha bir massivdagi tartib asosida boshqa bir bog'liq massivni qayta tartiblash kerak bo'lganda qo'llaniladi.

Misol:

```
scores = np.array([88, 95, 70, 85])
students = np.array(["Anish", "Rahul", "Priya", "Vikram"])
```

```
# Ballar bo'yicha saralangan indekslarni olamiz
sorted_indices = np.argsort(scores)

# Ushbu indekslar yordamida talabalar ismlarini tartiblaymiz
sorted_students = students[sorted_indices]

print(sorted_students)

['Priya' 'Vikram' 'Anish' 'Rahul']
```

3. `numpy.lexsort()`

Ushbu funksiya bir nechta saralash kalitlaridan foydalangan holda bilvosita barqaror saralashni amalga oshiradi. Kiruvchi kortejdagi **oxirgi kalit** asosiy saralash kaliti sifatida ishlatiladi. Bu funksiya ayniqsa tuzilmaviy yoki jadval ko'rinishidagi ma'lumotlarni saralashda juda foydalidir.

Misol:

```
age = np.array([25, 35, 20, 30])
salary = np.array([50000, 60000, 45000, 55000])

# Avval yosh (age) bo'yicha saralaydi (chunki kortejda oxirgi turibdi)
sorted_indices = np.lexsort((salary, age))

print(sorted_indices)

[2 0 3 1]
```

Izoh: Natijadagi `[2 0 3 1]` indekslari yoshning o'sib borish tartibini bildiradi: 20 (indeks 2), 25 (indeks 0), 30 (indeks 3) va 35 (indeks 1). Agar yoshlar teng bo'lib qolsa, u holda maosh (salary) qiymatlariga qarab tartiblanadi.

4. `ndarray.sort()`

Ushbu metod massivni o'z o'rnida (**in-place**) saralaydi, ya'ni asl massivning o'zi modifikatsiya qilinadi. Yangi massiv yaratilmagani sababli, bu usul xotira jihatidan juda samarali hisoblanadi.

Misol:

```
cities = np.array(["Delhi", "Mumbai", "Chennai", "Bangalore"])
cities.sort()

print(cities)

['Bangalore' 'Chennai' 'Delhi' 'Mumbai']
```

5. `numpy.sort_complex()`

`numpy.sort_complex()` funksiyasi kompleks sonlardan iborat massivni saralash uchun mo'ljallangan. U avval sonning haqiqiy (real) qismi bo'yicha saralaydi, agar haqiqiy qismlar teng bo'lsa, u holda mavhum (imaginary) qismi bo'yicha tartiblaydi.

Misol:

```
arr = np.array([3+2j, 1+5j, 3+1j])
sorted_arr = np.sort_complex(arr)

print(sorted_arr)

[1.+5.j 3.+1.j 3.+2.j]
```

Izoh: Natijada ko'rib turganingizdek, avval haqiqiy qismi eng kichik bo'lgan `1.+5.j` chiqdi. Keyin haqiqiy qismi bir xil (`3`) bo'lgan sonlar keldi va ular orasida mavhum qismi kichikrog'i (`1j`) birinchi bo'lib joylashdi.

6. `numpy.partition()`

`numpy.partition()` metodi massivni qisman saralaydi, ya'ni berilgan pozitsiyadagi element o'zining to'g'ri o'rniga joylashtiriladi. Undan kichik bo'lgan elementlar uning oldidan, kattalari esa undan keyin joylashadi, biroq butun massiv to'liq saralanishi kafolatlanmaydi.

Misol:

```
prices = np.array([1200, 500, 800, 300, 1500])
# 2-indeksdagi elementni o'z o'rniga qo'ygan holda qisman saralash
partitioned_prices = np.partition(prices, 2)

print(partitioned_prices)

[ 300  500  800 1200 1500]
```

7. `numpy.argpartition()`

`numpy.argpartition()` funksiyasi massivni berilgan pozitsiya atrofida qismlarga ajratuvchi **indekslarni** qaytaradi. Bu usul eng katta yoki eng kichik k ta elementni (top-k or bottom-k) samarali topish uchun juda foydalidir.

Misol:

```
marks = np.array([78, 92, 65, 88, 70])
# Eng katta 2 ta elementni ajratib olish uchun indekslarni aniqlash
top_indices = np.argpartition(marks, -2)
```

```
print(marks[top_indices])
```

```
[65 70 78 88 92]
```

Izoh: `np.argpartition(marks, -2)` funksiyasi massivning eng oxirgi ikkita o'rniga (eng katta qiymatlar uchun) mos indeksni joylashtiradi. Bu usul to'liq saralashga (`sort`) qaraganda ancha tezroq ishlaydi, chunki u barcha elementlarni tartibga solishga vaqt sarflamaydi, faqat kerakli qismni ajratib beradi.

Qidirish (Searching)

NumPy-da qidirish muayyan shart yoki taqqoslashga mos keladigan elementlarning o'rnini (indeksini) aniqlash uchun ishlatiladi. U 1-D va 2-D massivlar, satrlar, mantiqiy (boolean) qiymatlar va yetishmayotgan qiymatlarga ega massivlar kabi turli ma'lumotlar tuzilmalarini qo'llab-quvvatlaydi. Ushbu operatsiyalar katta ma'lumotlar to'plamida maksimal, minimal yoki mos keluvchi elementlarni samarali aniqlashga yordam beradi.

1. `numpy.argmax()`

`numpy.argmax()` funksiyasi massivdagi maksimal qiymatning **indeksini** qaytaradi. Biz qidiruvni butun massiv bo'ylab yoki ko'p o'lchamli ma'lumotlarda ma'lum bir o'q (axis) bo'ylab amalga oshirishimiz mumkin.

Misol:

```
# 2-D massiv yaratish (masalan, talabalar baholari)
marks = np.array([[78, 85, 90],
                  [88, 76, 95]])

print("Kiruvchi massiv:\n", marks)

# Butun massiv bo'ylab eng katta element indeksi
print("Umumiy maksimal indeks:", np.argmax(marks))

# Ustunlar bo'yicha (axis=0) maksimal indekslar
print("Ustunlar bo'yicha maksimal indeks:", np.argmax(marks, axis=0))

# Qatorlar bo'yicha (axis=1) maksimal indekslar
print("Qatorlar bo'yicha maksimal indeks:", np.argmax(marks, axis=1))
```


Kiruvchi massiv:

```
[[78 85 90]  
 [88 76 95]]
```

Umumiy maksimal indeks: 5

Ustunlar bo'yicha maksimal indeks: [1 0 1]

Qatorlar bo'yicha maksimal indeks: [2 2]

Izoh:

- **Umumiy maksimal indeks (5):** NumPy massivni yassilangan (flattened) deb tasavvur qiladi. 95 soni 5-indeksda joylashgan.
- **Ustunlar bo'yicha (`axis=0`):** Har bir ustundagi eng katta sonning qator indeksini qaytaradi. Masalan, birinchi ustunda 78 va 88 bor, 88 kattaroq va u 1-qatorda (indeks 1).
- **Qatorlar bo'yicha (`axis=1`):** Har bir qatordagi eng katta sonning ustun indeksini qaytaradi. Birinchi qatorda 90 eng katta va u 2-indeksda joylashgan.

2. `numpy.nanargmax()`

`numpy.nanargmax()` funksiyasi `argmax()` kabi ishlaydi, biroq qidiruv jarayonida **NaN** (mavjud bo'lmagan/son emas) qiymatlarni e'tiborsiz qoldiradi. Bu funksiya to'liq bo'lmagan yoki ma'lumotlari yetishmaydigan to'plamlar bilan ishlashda juda foydali.

Misol:

```
# NaN qiymatga ega bo'lgan massiv  
temperatures = np.array([np.nan, 32, 45, 40])  
print("Kiruvchi ma'lumot:", temperatures)  
  
# NaN ni hisobga olmasdan maksimal qiymat indeksini topish  
print("NaN dan tashqari maksimal indeks:", np.nanargmax(temperatures))  
  
# NaN qiymatga ega 2D massiv  
data = np.array([[np.nan, 25],  
                 [30, 28]])  
print("2D massiv:\n", data)  
  
# Qatorlar bo'yicha (axis=1) NaN ni hisobga olmasdan maksimal indekslar  
print("Qatorlar bo'yicha maksimal indeks:", np.nanargmax(data, axis=1))
```

Kiruvchi ma'lumot: [nan 32. 45. 40.]
NaN dan tashqari maksimal indeks: 2
2D massiv:
[[nan 25.]
[30. 28.]]
Qatorlar bo'yicha maksimal indeks: [1 0]

Izoh:

- **temperatures massivda:** Oddiy `argmax()` funksiyasi NaN qiymatini ko'rganda xatolik berishi yoki noto'g'ri natija qaytarishi mumkin, lekin `nanargmax()` uni shunchaki tashlab ketadi va 45 sonining indeksi bo'lgan `2` ni qaytaradi.
- **2D massivda:** Birinchi qatorda `[nan, 25]` berilgan. `nanargmax` NaN ni o'tkazib yuboradi va yagona qolgan qiymat `25` ning indeksi `1` ni qaytaradi. Ikkinchi qatorda esa `30 > 28` bo'lgani uchun indeks `0` ni qaytaradi.

3. `numpy.argmin()`

`numpy.argmin()` funksiyasi massivdagi eng kichik (minimal) qiymatning **indeksini** qaytaradi. Bu funksiya eng kichik elementning o'rnini tez va samarali aniqlashga yordam beradi.

Misol:

```
prices = np.array([1200, 850, 999, 650])
print("Narxlar:", prices)

# Eng arzon narxning indeksini topish
print("Eng past narx indeksi:", np.argmin(prices))
```

Narxlar: [1200 850 999 650]
Eng past narx indeksi: 3

4. `numpy.nanargmin()`

`numpy.nanargmin()` funksiyasi **NaN** (mavjud bo'lmagan) qiymatlarni hisobga olmagan holda massivdagi minimal qiymat indeksini qaytaradi. Bu ma'lumotlar to'plamida bo'sh kataklar yoki yetishmayotgan yozuvlar bo'lganda ishonchli natija olishni ta'minlaydi.

Misol:

```
scores = np.array([np.nan, 78, 65, 80])
print("Ballar:", scores)
```

```
# NaN ni e'tiborsiz qoldirib, minimal qiymat indeksini topish
print("NaN dan tashqari minimal indeks:", np.nanargmin(scores))
```

Ballar: [nan 78. 65. 80.]

NaN dan tashqari minimal indeks: 2

Izoh: `nanargmin` funksiyasi bo'lmaganda, oddiy `argmin` funksiyasi NaN qiymatini eng kichik deb hisoblashi yoki natijada xatolik (warning) berishi mumkin edi. `nanargmin` esa 65 sonini eng kichik deb topadi va uning indeksi bo'lgan 2 ni qaytaradi.

5. `numpy.argwhere()`

`numpy.argwhere()` funksiyasi berilgan shartga mos keladigan elementlarning indekslarini qaytaradi. Natija har bir element bo'yicha guruhlanadi (ustun ko'rinishida), bu esa ma'lumotlarni filtrlash uchun juda qulaydir.

Misol:

```
arr = np.array([10, 0, -5, 0, 8])
# 0 dan katta elementlar indeksini topish
indices = np.argwhere(arr > 0)

print(indices)
```

```
[[0]
 [4]]
```

6. `numpy.nonzero()`

`numpy.nonzero()` funksiyasi massivdagi nolga teng bo'lmagan barcha elementlarning indekslarini qaytaradi. U ko'pincha shartga asoslangan tezkor qidiruvlar uchun ishlatiladi.

Misol:

```
values = np.array([10, 0, 20, 0, 30])

# Nolga teng bo'lmagan elementlar indeksini topish
print("Nolga teng bo'lmagan indekslar:", np.nonzero(values))
```

Nolga teng bo'lmagan indekslar: (array([0, 2, 4]),)

Izoh:

- `argwhere` natijani 2D massiv ko'rinishida qaytaradi, bu esa har bir topilgan indeksni alohida qator sifatida ko'rish imkonini beradi.

- `nonzero` esa natijani kortej (tuple) ichida qaytaradi.
`np.nonzero(a)` aslida `np.where(a != 0)` bilan bir xil ishlaydi.
- Agar sizda ko'p o'lchamli massiv bo'lsa, `nonzero` har bir o'lcham uchun alohida massivlar kortejini qaytaradi, bu esa o'sha elementlarning koordinatalarini bildiradi.

7. `numpy.flatnonzero()`

`numpy.flatnonzero()` funksiyasi massivni yassilangan (1-D) deb hisoblagan holda, uning nolga teng bo'lmagan elementlari indekslarini qaytaradi. Bu, ayniqsa, ko'p o'lchamli massivlar bilan ishlayotganda va ularni bitta qatorli indekslar orqali boshqarish kerak bo'lganda juda qo'l keladi.

Misol:

```
matrix = np.array([[0, 1], [2, 0]])

# Yassilangan holatdagi nol bo'lmagan indekslarni topish
print("Yassilangan nol bo'lmagan indekslar:", np.flatnonzero(matrix))
```

Yassilangan nol bo'lmagan indekslar: [1 2]

8. `numpy.where()`

`numpy.where()` funksiyasi berilgan shartga asoslangan elementlarni qaytaradi. U ko'pincha ma'lumotlarni shartli ravishda o'zgartirish yoki ikkita massivdan qiymatlarni tanlab olish uchun ishlatiladi.

Misol:

```
marks = np.array([45, 72, 88, 60])

# Shart: agar baho 60 dan katta yoki teng bo'lsa "Pass", aks holda "Fail"
result = np.where(marks >= 60, "Pass", "Fail")

print("Natija:", result)
```

Natija: ['Fail' 'Pass' 'Pass' 'Pass']

Izoh:

- `flatnonzero` : Yuqoridagi misolda `matrix` yassilansa `[0, 1, 2, 0]` ko'rinishiga keladi. Nolga teng bo'lmagan `1` va `2` sonlari mos ravishda `1` va `2` -indekslarda joylashgan.
- `where` : Bu funksiya "if-else" mantiqi bo'yicha butun massiv ustida bir marta amallarni bajarishga imkon beradi (vectorization),

bu esa oddiy sikllarga qaraganda ancha tezroqdir.

9. `numpy.searchsorted()`

`numpy.searchsorted()` funksiyasi tartiblangan massivda ma'lum bir qiymatning tartibni buzmaganda qayerga joylashtirilishi kerakligini (indeksini) aniqlaydi. Bu funksiya asosan ikkilik qidiruv (binary search) algoritmlarida qo'llaniladi.

Misol:

```
sorted_scores = np.array([40, 55, 70, 85])

# 65 soni tartibni saqlash uchun qaysi indeksga qo'shilishi kerak?
print("65 uchun joylashtirish o'rni:", np.searchsorted(sorted_scores, 65))

65 uchun joylashtirish o'rni: 2
```

10. `numpy.extract()`

`numpy.extract()` funksiyasi berilgan shartga mos keladigan elementlarning o'zini qaytaradi. Bu funksiya ma'lumotlarni to'g'ridan-to'g'ri filtrlash va ajratib olish uchun juda qulaydir.

Misol:

```
ages = np.array([12, 18, 25, 15, 30])

# 18 va undan katta yoshdagilarni ajratib olish
adults = np.extract(ages >= 18, ages)

print("Kattalar:", adults)

Kattalar: [18 25 30]
```

Izoh:

- **`searchsorted`** : 65 soni 55 va 70 ning orasiga tushishi kerak, shuning uchun u 2-indeksni qaytaradi. Shuni yodda tutingki, ushbu funksiya massivning **saralangan** bo'lishini talab qiladi.
- **`extract`** : Bu funksiya mantiqiy shart (`ages >= 18`) asosida ishlaydi. U `np.where` funksiyasiga o'xshash, lekin indekslarni emas, balki qiymatlarning o'zini massiv ko'rinishida qaytaradi.

Hisoblash (Counting)

NumPy-da hisoblash amallari muayyan shartga mos keladigan elementlar sonini (masalan, nolga teng bo'lmagan qiymatlar, mantiqiy mosliklar yoki takrorlanish chastotasi) aniqlash uchun ishlatiladi. Ushbu operatsiyalar ma'lumotlarni samarali umumlashtirish va massivlardan muhim statistik tushunchalarni ajratib olishga yordam beradi. NumPy-ning optimallashtirilgan hisoblash mexanizmlari ham bir o'lchamli, ham ko'p o'lchamli ma'lumotlar bilan juda yaxshi ishlaydi.

1. `numpy.count_nonzero()`

`numpy.count_nonzero()` funksiyasi massivdagi nolga teng bo'lmagan elementlar sonini qaytaradi. U sonli qiymatlar, mantiqiy (boolean) ifodalar va shartli amallar bilan ishlay oladi.

Misol:

```
arr = np.array([0, 10, 0, 25, 30, 0])
count = np.count_nonzero(arr)

print("Massiv:", arr)
print("Nol bo'lmaganlar soni:", count)
```

```
Massiv: [ 0 10  0 25 30  0]
Nol bo'lmaganlar soni: 3
```

Izoh: Bu funksiya nafaqat sonlarni, balki shartli hisoblashlarni ham bajara oladi. Masalan, `np.count_nonzero(arr > 15)` deb yozsangiz, massivdagi 15 dan katta bo'lgan elementlar sonini qaytaradi. Bu NumPy-da ma'lum bir shartga mos keladigan elementlar sonini topishning eng tezkor usulidir.

2. `numpy.bincount()`

`numpy.bincount()` funksiyasi massivdagi manfiy bo'lmagan butun sonlarning uchrash sonini hisoblaydi. Natijadagi har bir indeks butun son qiymatini, o'sha indeksdagi qiymat esa uning necha marta takrorlanganini bildiradi.

Misol:

```
numbers = np.array([1, 2, 2, 3, 3, 3, 4, 5, 10, 5])
frequency_counts = np.bincount(numbers)

print("Har bir sonning takrorlanish chastotasi:")
for value, count in enumerate(frequency_counts):
```

```
if count > 0:  
    print(f"{value} soni {count} marta qatnashgan")
```

Har bir sonning takrorlanish chastotasi:

```
1 soni 1 marta qatnashgan  
2 soni 2 marta qatnashgan  
3 soni 3 marta qatnashgan  
4 soni 1 marta qatnashgan  
5 soni 2 marta qatnashgan  
10 soni 1 marta qatnashgan
```

3. `numpy.unique()`

`numpy.unique()` funksiyasi massivdagi takrorlanmas (unikal) elementlarni topish uchun ishlatiladi. Shuningdek, u har bir unikal qiymatning chastotasi, birinchi marta uchrash indeksleri kabi qo'shimcha ma'lumotlarni ham qaytarishi mumkin. Bu uni ma'lumotlar tahlili uchun juda foydali qiladi.

Misol:

```
data = np.array(["apple", "banana", "apple", "orange", "banana", "apple",  
# Unikal elementlar va ularning sonini aniqlash  
unique_items, counts = np.unique(data, return_counts=True)  
  
print("Takrorlanmas elementlar:", unique_items)  
print("Ular soni:", counts)
```

Takrorlanmas elementlar: ['apple' 'banana' 'orange']

Ular soni: [3 2 1]

Izoh:

- **`bincount`** : Faqat manfiy bo'lmagan butun sonlar (`int`) bilan ishlaydi. Agar massivda 10 soni bo'lsa, natijaviy massiv uzunligi kamida 11 bo'ladi (chunki 0 dan 10 gacha indekslar bo'lishi kerak).
- **`unique`** : Istalgan turdagi ma'lumotlar (matn, suzuvchi nuqtali sonlar va h.k.) bilan ishlay oladi. `return_counts=True` parametri orqali har bir element necha marta kelganini osongina bilish mumkin.



Nazorat savollari

1-daraja: Asosiy tushunchalar

1. NumPy-da saralash (`sorting`) tushunchasi nimani anglatadi va u odatda qaysi tartibda amalga oshiriladi?

2. `np.sort()` funksiyasi yordamida massivni kamayish tartibida saralash uchun qanday usuldan foydalaniladi?
3. Massivni o'z o'rnida (`in-place`) saralash bilan yangi nusxa yaratib saralashning asosiy farqi nimada?
4. Python-ning standart `sorted()` funksiyasi NumPy massivi bilan ishlatilganda qanday turdagi ma'lumot qaytaradi?

2-daraja: Funksiyalarning ishlash mexanizmi

5. `np.sort()` funksiyasida `axis=0` va `axis=-1` parametrlari 2D massivda saralash yo'nalishini qanday belgilaydi?
6. `np.argsort()` funksiyasi qiymatlarning o'zini emas, indekslarni qaytarishi amaliyotda qanday qulaylik yaratadi?
7. `np.lexsort()` funksiyasida bir nechta kalitlar bilan saralashda qaysi massiv asosiy kalit vazifasini bajaradi?
8. `np.searchsorted()` funksiyasi ishlashi uchun kiruvchi massiv qanday holatda bo'lishi talab etiladi?

3-daraja: Amaliy tahlil va murakkab amallar

9. `np.where()` funksiyasidan faqat shart berilgan holatda foydalanilsa, u qanday natija qaytaradi?
10. `np.partition()` va `np.argpartition()` funksiyalari to'liq saralashdan ko'ra qaysi jihati bilan samaraliroq hisoblanadi?
11. NaN (mavjud bo'lmagan) qiymatlarga ega massivlarda maksimal indeksni topishda `np.argmax()` va `np.nanargmax()` farqini tushuntiring.
12. `np.bincount()` va `np.unique()` funksiyalari elementlarni sanashda qanday cheklovlar yoki afzalliklarga ega?